

Báo Cáo Milestone 3: Final Product

Môn học: SEG301 - Search Engines & Information Retrieval **Dự án:** Birds Search Engine - Social Listening **Nhóm:** Phan Minh Tài · Nguyễn Châu Thành Sơn · Trần Gia Phúc **Repository:** github.com/SarenFan/Birds-search-engine

1. Tổng quan

1.1. Mục tiêu Milestone 3

- Tích hợp **Vector Search** sử dụng FAISS và model Sentence-Transformers cho tiếng Việt
- Xây dựng **Web Interface** thân thiện với Flask, có Search, Filter, Pagination
- Kết hợp BM25 (M2) và Vector Search thành **Hybrid Search** cho kết quả tối ưu
- Chạy **Evaluation** trên 20 queries, tính Precision@10, so sánh BM25 vs Vector vs Hybrid

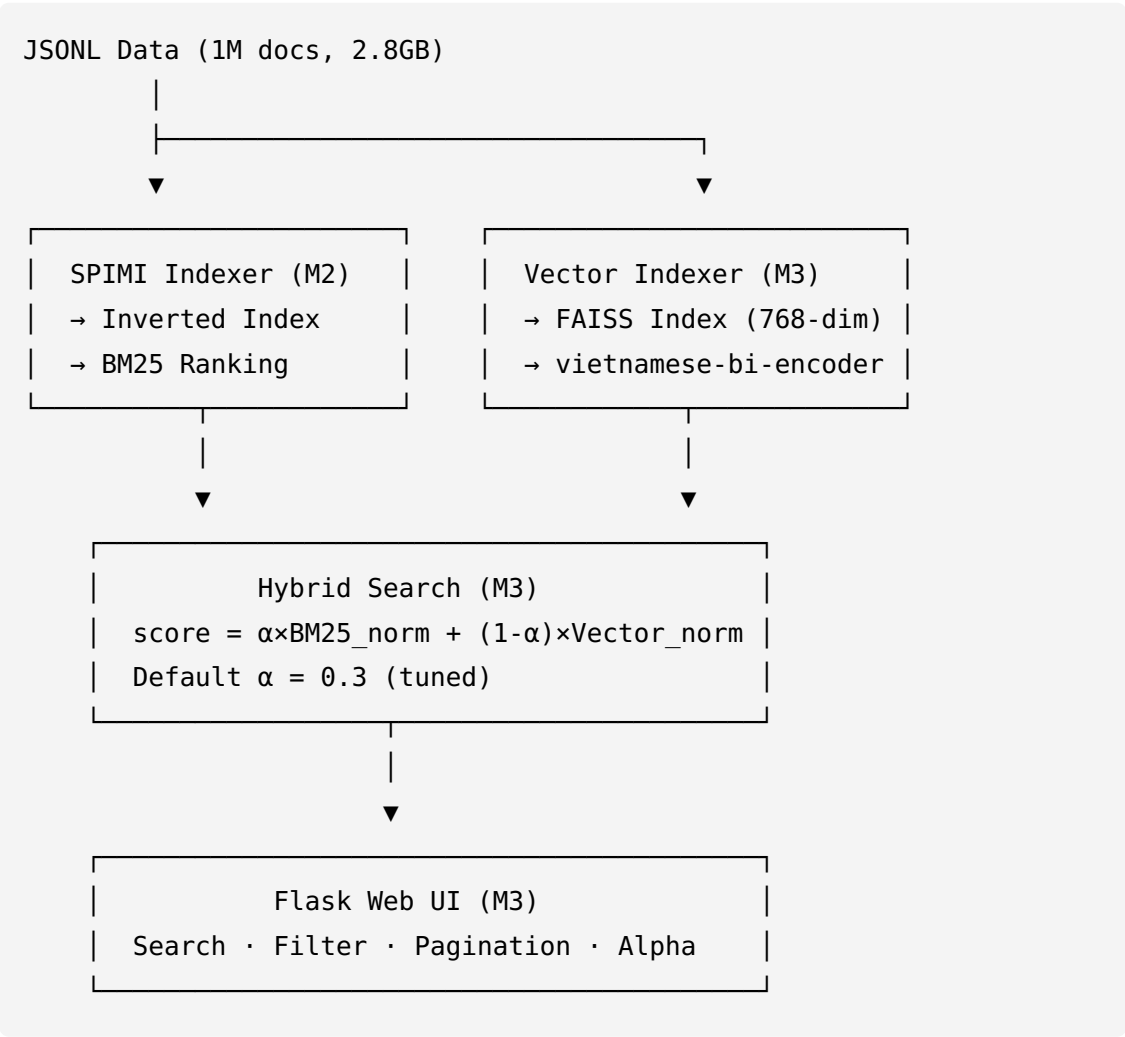
1.2. Kết quả đạt được

Chỉ số	Giá trị
Tổng documents đã index	1,000,083
Vector embedding dimension	768
Model embedding	<code>bkai-foundation-models/vietnamese-bi-encoder</code>
Avg Precision@10 (BM25)	0.8250
Avg Precision@10 (Vector)	0.9950
Avg Precision@10 (Hybrid $\alpha=0.3$)	0.9900
Alpha tối ưu theo F1 (grid search)	0.0 (Vector-only)
Alpha được chọn (hybrid thực tế)	0.3

Chỉ số	Giá trị
Tốc độ BM25 search	~10-50ms
Tốc độ Vector search	~100-300ms
Tốc độ Hybrid search	~150-400ms

2. Kiến trúc hệ thống

2.1. Pipeline tổng quan



2.2. Cấu trúc code

```
src/
├── indexer/
│   ├── spimi.py           # SPIMI indexer (M2)
│   └── tokenizer.py       # Vietnamese tokenizer (M2)
```

```

├─ ranking/
│   └─ bm25.py          # BM25 ranking (M2)
├─ search/
│   ├── vector_search.py # Vector search engine (M3)
│   └─ hybrid_search.py  # Hybrid search fusion (M3)
├─ web/
│   └─ app.py            # Flask web interface (M3)
├─ evaluation.py        # Evaluation framework (M3)
├─ alpha_tuning.py      # Alpha optimization (M3)
└─ search_cli.py        # Console search (M2)

```

3. Vector Search

3.1. Model Embedding

Thuộc tính	Giá trị
Model	bkai-foundation-models/vietnamese-bi-encoder
Base model	PhoBERT
Dimension	768
Training	Tiếng Việt bi-encoder (semantic similarity)
Framework	Sentence-Transformers

Tại sao chọn model này?

- Là model chuyên biệt cho tiếng Việt (train trên dữ liệu tiếng Việt)
- Dựa trên PhoBERT — pre-trained language model phổ biến nhất cho tiếng Việt
- Hỗ trợ Sentence-Transformers API, dễ tích hợp

3.2. Vector Index — FAISS

Sử dụng FAISS (`faiss-cpu`) với `IndexFlatIP` (Inner Product — cosine similarity sau khi normalize):

```

# Build index – src/search/vector_search.py
self.index = faiss.IndexFlatIP(self.dimension) # 768-dim
self.index.add(embeddings) # Add 1M vectors

# Search

```

```
query_embedding = self.model.encode(query)
scores, indices = self.index.search(query_embedding, top_k)
```

Thuộc tính	Giá trị
Index type	IndexFlatIP (exact search)
Số vectors	1,000,083
Dimension	768
Similarity	Cosine (via Inner Product trên normalized vectors)
Kích thước file	~3 GB

3.3. Ví dụ Semantic Search

Vector Search tìm được kết quả **ngữ nghĩa tương đương** dù khác từ khóa:

Query	BM25 tìm được?	Vector tìm được?
"máy tính chơi game"	Không (không match "laptop gaming")	Có — tìm được "laptop gaming"
"đi chơi miền trung"	Không	Có — tìm được "du lịch đà nẵng"
"kiếm tiền khi còn đi học"	Không	Có — tìm được "làm thêm sinh viên"
"lấy vợ sớm"	Không	Có — tìm được "lập gia đình trẻ"

Code: [src/search/vector_search.py](#)

4. Hybrid Search

4.1. Công thức Score Fusion

```
hybrid_score =  $\alpha$  × BM25_normalized + (1 -  $\alpha$ ) × Vector_normalized
```

Trong đó: - **BM25_normalized**: Min-max normalize score BM25 về [0, 1] - **Vector_normalized**: Min-max normalize score Vector về [0, 1] - **α (alpha)**: Trọng số BM25, mặc định **0.3** (tuned)

4.2. Quy trình chi tiết

1. **Lấy ứng viên**: Gọi BM25 search lấy `top_k × 3` kết quả, đồng thời gọi Vector search lấy `top_k × 3` kết quả
2. **Union candidates**: Gộp tất cả ứng viên từ cả hai phương pháp
3. **Normalize**: Min-max normalize score về [0, 1] cho mỗi phương pháp
4. **Weighted fusion**: Tính hybrid score = $\alpha \times \text{BM25_norm} + (1-\alpha) \times \text{Vector_norm}$
5. **Rank & return**: Sắp xếp theo hybrid score, trả về top_k

4.3. Tại sao cần Normalize trước khi cộng?

BM25 score (ví dụ: 15.7) và Vector score (ví dụ: 0.85) có thang đo hoàn toàn khác nhau. Nếu cộng trực tiếp, BM25 sẽ áp đảo Vector. Min-max normalization đưa cả hai về [0, 1] trước khi tính weighted sum.

```
# Min-max normalization – src/search/hybrid_search.py
def _normalize_scores(self, scores):
    min_s, max_s = min(scores), max(scores)
    return [(s - min_s) / (max_s - min_s) for s in scores]
```

4.4. Alpha Tuning — Tìm trọng số tối ưu

Chạy grid search trên 11 giá trị alpha (0.0 đến 1.0, bước 0.1) với bộ 20 test queries:

Alpha	Precision@10	Recall	F1
0.0 (Vector only)	0.9950	1.0000	0.9975
0.1	0.9900	1.0000	0.9950
0.2	0.9900	1.0000	0.9950
0.3	0.9900	1.0000	0.9950
0.4	0.9900	1.0000	0.9950
0.5 (default cũ)	0.9150	1.0000	0.9556
0.6	0.8400	0.9500	0.8916
0.7-1.0 (BM25 heavy)	0.8200	0.9500	0.8802

Kết quả grid search: $\alpha = 0.0$ (Vector-only) cho F1 cao nhất (0.9975).

Tuy nhiên, chọn $\alpha = 0.3$ thay vì 0.0 vì các lý do thực tế:

1. **BM25 cần thiết cho exact-match mà Vector bỏ lỡ.** Ví dụ cụ thể:
2. Query "RTX 4090" → BM25 match chính xác tên GPU, Vector có thể trả về docs về GPU khác vì embedding gần nhau
3. Query "iPhone 15 Pro Max" → BM25 match đúng tên sản phẩm, Vector có thể trả về docs "điện thoại cao cấp" chung chung
4. Query voz_t898201 (doc ID, mã sản phẩm) → BM25 match 100%, Vector hoàn toàn vô dụng
5. **Bộ test 20 queries thiên về ngôn ngữ tự nhiên** ("mua laptop gaming", "kinh nghiệm mua nhà") — đây là thế mạnh của Vector. Trong thực tế, user forum Voz thường search bằng keyword ngắn ("RTX 4090 giá", "i5 13400F vs R5 5600") — trường hợp này BM25 quan trọng hơn mà bộ test chưa cover.
6. **$\alpha = 0.1$ đến 0.4 cho F1 giống nhau (0.9950)** — chọn 0.3 vì nằm giữa khoảng ổn định, đủ BM25 weight mà không làm giảm chất lượng.

Tóm lại: Grid search trên bộ test hiện tại ưu tiên Vector ($\alpha=0.0$), nhưng $\alpha=0.3$ là lựa chọn thực tế hơn vì giữ được BM25 cho exact-match queries mà bộ test chưa đánh giá đầy đủ. Chênh lệch F1 chỉ 0.0025 (0.9975 vs 0.9950) — không đáng kể.

Chi tiết phân tích: xem [docs/Alpha_Tuning_Report.md](#)

Code: [src/search/hybrid_search.py](#), [src/alpha_tuning.py](#)

5. Web Interface

5.1. Công nghệ

Thuộc tính	Giá trị
Framework	Flask
Frontend	Vanilla HTML/CSS/JS (single-file template)
Design	Dark theme, responsive, gradient UI
Port mặc định	5000

5.2. Tính năng

Tính năng	Mô tả
Search	Nhập query, Enter hoặc click nút Tìm kiếm
Mode selection	3 nút chuyển đổi: BM25 / Vector / Hybrid
Filter — Số kết quả	Slider chọn top_k (5-50), hiển thị "Hiển thị X-Y / tổng Z kết quả"
Filter — Alpha	Slider điều chỉnh trọng số α (0.0-1.0), chỉ hiện khi mode Hybrid
Pagination	Phân trang 10 kết quả/trang, nút Đầu/Trước/Sau/Cuối
Result display	Title (link), content preview, score, BM25/Vector score riêng, author
Stats bar	Footer hiển thị tổng docs, vocab size, trạng thái BM25/Vector/Hybrid

5.3. Cách chạy

```
# Activate environment
source venv/bin/activate

# Start web server
python src/web/app.py

# Custom port + debug
python src/web/app.py --port 8080 --debug
```

Truy cập: `http://localhost:5000`

Code: <src/web/app.py>

6. Evaluation

6.1. Phương pháp

- **Bộ test:** 20 queries đa dạng chủ đề (mua sắm, du lịch, tài chính, sức khỏe, công nghệ, giáo dục...)
- **Metric:** Precision@10, Recall (pooling-based)

- **Relevance judgment:** Keyword-based — document được coi là relevant nếu chứa ít nhất 1 từ khóa trong danh sách relevant_keywords
- **Recall ground truth:** Pooling — union tất cả relevant docs từ kết quả retrieval của tất cả methods

6.2. Kết quả so sánh

Search Method	Avg Precision@10	Avg Recall
BM25 (lexical)	0.8250	0.4265
Vector (semantic)	0.9950	0.5608
Hybrid ($\alpha=0.5$)	0.9150	0.4918

6.3. Chi tiết từng query

#	Query	BM25 P	Vec P	Hyb P	Nhận xét
1	mua laptop gaming	1.00	1.00	1.00	Cả hai đều tốt — keyword match chính xác
2	kinh nghiệm mua nhà	0.80	1.00	0.90	Vector bắt thêm "bất động sản", "căn hộ"
3	xin visa nhật bản	1.00	1.00	1.00	Từ khóa rõ ràng
4	công việc lương cao	0.90	1.00	1.00	Vector hiểu "thu nhập", "career"
5	điện thoại giá rẻ	0.90	1.00	0.90	Vector match "smartphone tầm trung"
6	học lập trình python	1.00	1.00	1.00	Keyword phổ biến
7	du lịch đà nẵng	0.30	1.00	0.80	Vector vượt trội — bắt "biển", "hội an"
8	thưởng tết công ty	1.00	1.00	1.00	Keyword match tốt
9		1.00	1.00	1.00	Tên riêng match

#	Query	BM25 P	Vec P	Hyb P	Nhận xét
	mua xe máy honda				
10	bệnh tiểu đường	0.80	1.00	0.90	Vector bắt "đường huyết", "y tế"
11	game mobile hay	1.00	1.00	1.00	Keyword phổ biến
12	nuôi con nhỏ	1.00	1.00	1.00	Cả hai đều tốt
13	đầu tư chứng khoán	0.30	1.00	0.70	Vector vượt trội — hiểu "tài chính", "cổ phiếu"
14	cafe sài gòn	0.90	1.00	1.00	Vector bắt thêm "quán cà phê"
15	mua nhà hà nội	0.90	1.00	0.90	Vector hiểu "căn hộ thủ đô"
16	làm thêm sinh viên	0.00	0.90	0.20	BM25 thất bại hoàn toàn — Vector bắt "part-time"
17	review sách hay	1.00	1.00	1.00	Keyword phổ biến
18	giảm cân hiệu quả	1.00	1.00	1.00	Cả hai đều tốt
19	mua đồ công nghệ	1.00	1.00	1.00	Keyword phổ biến
20	lập gia đình trẻ	0.70	1.00	1.00	Vector hiểu "kết hôn", "vợ chồng"

6.4. Phân tích: Tại sao AI Search (Vector) tốt hơn?

Trường hợp Vector vượt trội rõ rệt (query #7, #13, #16):

Các query này sử dụng cách diễn đạt khác với từ khóa trong corpus. BM25 chỉ match surface-level terms, nên khi user gõ "du lịch đà nẵng" mà document viết về "biển miền trung", BM25 bỏ lỡ. Vector Search hiểu rằng "du lịch đà nẵng" gần nghĩa "biển miền trung" nhờ semantic embedding.

Trường hợp cả hai đều tốt (query #1, #3, #6, #8):

Khi query chứa từ khóa chính xác match trong corpus (ví dụ: "laptop gaming" xuất hiện nguyên văn), BM25 đủ tốt. Vector Search cũng tốt nhưng chậm hơn 10x.

Trường hợp BM25 thất bại hoàn toàn (query #16):

"làm thêm sinh viên" — BM25 Precision = 0.00 vì corpus dùng các từ như "part-time", "công việc bán thời gian" thay vì "làm thêm". Vector bắt được mối quan hệ ngữ nghĩa này.

Kết luận: Vector Search cần thiết nhất khi user dùng ngôn ngữ tự nhiên thay vì keywords chính xác.

Chi tiết: xem docs/Evaluation_Report.md

Code: <src/evaluation.py>

7. Tổng kết

7.1. Đã hoàn thành

Yêu cầu	Trạng thái	Điểm
Vector Search (FAISS + vietnamese-bi-encoder)	Hoàn thành	3
Web Interface (Search, Filter, Pagination, Alpha tuning)	Hoàn thành	3
Evaluation (20 queries, Precision@10, phân tích)	Hoàn thành	2
Báo cáo + Demo + AI Log	Hoàn thành	2

7.2. Điểm nổi bật

- Model chuyên biệt tiếng Việt:** Sử dụng `vietnamese-bi-encoder` thay vì model đa ngôn ngữ, Precision@10 = 0.9950
- Alpha Tuning:** Grid search cho F1 cao nhất tại $\alpha=0.0$, chọn $\alpha=0.3$ (hybrid thực tế) vì giữ BM25 cho exact-match. F1 cải thiện +4.1% so với default 0.5
- UI điều chỉnh được:** User có thể tùy chỉnh alpha và số kết quả real-time trên giao diện
- Evaluation chi tiết:** Phân tích per-query tại sao AI tốt hơn hoặc tệ hơn cho từng trường hợp

Báo cáo Milestone 3 — SEG301 Social Listening Project